



Mathematics of Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Online Scheduling with Known Arrival Times

Nicholas G. Hallhttp://fisher.osu.edu/~hall_33, Marc E. Posnerwww.iwse.osu.edu/ISEFaculty/Posner/Posner.htm, Chris N. Pottswww.soton.ac.uk/math/people/profiles/or/cnp1.html,

To cite this article:

Nicholas G. Hallhttp://fisher.osu.edu/~hall_33, Marc E. Posnerwww.iwse.osu.edu/ISEFaculty/Posner/Posner.htm, Chris N. Pottswww.soton.ac.uk/math/people/profiles/or/cnp1.html, (2009) Online Scheduling with Known Arrival Times. Mathematics of Operations Research 34(1):92-102. <https://doi.org/10.1287/moor.1080.0346>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2009, INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Online Scheduling with Known Arrival Times

Nicholas G. Hall

Department of Management Sciences and 3D Laboratory, The Ohio State University, Columbus, Ohio 43210,
hall_33@fisher.osu.edu, http://fisher.osu.edu/~hall_33

Marc E. Posner

Department of Integrated Systems Engineering and 3D Laboratory, The Ohio State University, Columbus, Ohio 43210,
posner.1@osu.edu, www.iwse.osu.edu/ISEFaculty/Posner/Posner.htm

Chris N. Potts

School of Mathematics, University of Southampton, Southampton SO17 1BJ, United Kingdom,
C.N.Potts@soton.ac.uk, www.soton.ac.uk/math/people/profiles/or/cnp1.html

We consider an online scheduling environment where decisions are made without knowledge of the data of jobs that may arrive later. However, additional jobs can only arrive at known future times. This environment interpolates between the classical offline and online scheduling environments and approaches the classical online environment when there are many equally spaced potential job arrival times. The objective is to minimize the sum of weighted completion times, a widely used measure of work-in-process inventory cost and customer service. For a nonpreemptive single machine environment, we show that a lower bound on the competitive ratio of any online algorithm is the solution of a mathematical program. This lower bound is between $(1 + \sqrt{5})/2$ and 2, with the exact value depending on the potential job arrival times. We also provide a “best possible” online scheduling algorithm and show that its competitive ratio matches this lower bound. We analyze two practically motivated special cases where the potential job arrival times have a special structure. When there are many equally spaced potential job arrival times, the competitive ratio of our online algorithm approaches the best possible competitive ratio of 2 for the classical online problem.

Key words: machine scheduling; online algorithm; total weighted completion time; competitive analysis

MSC2000 subject classification: Primary: 90B35; secondary: 68M20, 90C59

OR/MS subject classification: Primary: production/scheduling, sequencing, deterministic, single machine; secondary: production/scheduling, approximations/heuristic

History: Received October 12, 2006; revised October 4, 2007, January 13, 2008 and April 28, 2008.

1. Introduction. We consider a planning environment where some jobs are currently available and others may arrive in the future. The data for the currently available jobs are known. New jobs may arrive only at known discrete future times. There is no restriction on the data for the new jobs, which become known only when they arrive. This problem is an example of a semi-online planning problem where potential arrival times effectively define planning periods. For this reason, we consider our problem to be an *online planning period scheduling problem*.

In classical offline scheduling, all data are known at the start of the planning horizon, whereas in classical online scheduling, jobs may arrive at any time and their data become known only on arrival. Therefore, the environment we consider interpolates between the classical offline and online scheduling environments. Moreover, if the number of arrival times is large and uniformly distributed, then this new planning environment approximates the classical online environment as a limiting case. See Pruhs et al. [9] for a comprehensive survey of the online scheduling literature.

Our planning environment models the periodic ordering and scheduling processes that are widely used in industry (Reid and Sanders [10], Russell and Taylor [11]). Orders are accepted, for example, weekly. Between two orders, no new jobs become available. Until an order arrives, the details of the order, for example, the amount of processing it requires and its value, are not known. Meanwhile, it is necessary to make scheduling decisions about the currently available jobs, considering the possibility of new jobs arriving at the start of subsequent weeks. In order to have resources available to process new jobs that may arrive, it may be advantageous to insert idle time into the schedule.

We consider a single machine scheduling problem where the objective is to minimize the total weighted completion time of the jobs. This widely used scheduling objective is a proxy measure for work-in-process inventory cost in a make-to-stock environment (Pinedo [7]). It is also a proxy measure for customer service in a make-to-order environment (Chen and Hall [3]).

The best competitive ratio for the classical online single machine unweighted completion time problem is 2 (see Hoogeveen and Vestjens [5] and Phillips et al. [6]), which is also the best possible result. Anderson and Potts [1] generalize these results to the weighted problem. By creating a procedure that depends on the times when jobs may arrive, we resolve the question of whether it is possible to design an online algorithm with a competitive

ratio that is better than 2 for the weighted online planning period scheduling problem. In our proof technique, we first show that a lower bound on the competitive ratio of any online algorithm is given by the optimal solution of a mathematical programming problem. This lower bound is between $(1 + \sqrt{5})/2$ and 2, depending on the potential job arrival times. A specific online algorithm is then proposed. We prove that the online algorithm has a competitive ratio that matches the lower bound. Consequently, this algorithm is best possible.

This paper, therefore, makes two contributions. First, we propose an online scheduling environment in which jobs can arrive only at known future times, modeling typical planning processes in industry. Second, within this framework, we present a competitive analysis for the single machine total weighted completion time problem. Similar analysis may be possible for other scheduling problems in this new environment.

This paper is organized as follows. Section 2 provides notation, a problem definition, and some preliminary results. Section 3 describes a lower bound on the competitive ratio of any algorithm for our online planning period scheduling problem. Section 4 describes an online scheduling algorithm. In §5, we show that this algorithm has a competitive ratio that equals the lower bound and is, therefore, best possible. Section 6 contains a discussion of two practically motivated cases where the potential job arrival times have a special structure. Finally, §7 contains a conclusion and some suggestions for future research.

2. Preliminaries. In this section, we give a formal description of the problem and present some basic results. We consider an online planning period scheduling problem in which jobs arrive over time and are processed nonpreemptively on a single machine. The potential arrival times of jobs are denoted by $\mathcal{T} = \{t_0, t_1, \dots, t_T\}$, where $0 = t_0 < t_1 < \dots < t_T$. These times are known at the start of the planning horizon. We define $t_{T+1} = \infty$.

At time t_u , a set of jobs N_u is released for $u = 0, 1, \dots, T$, where N_u may be empty. Each job $j \in N_u$ has a release date $r_j = t_u$, a processing time $p_j > 0$, and a weight $w_j \geq 0$. Information about the jobs of N_u is known only at time t_u when the jobs become available. Therefore, scheduling decisions before time t_u are made without knowledge of the jobs in sets $N_u, \dots, N_T$. Let $N = \bigcup_{i=0}^T N_i$ and $n = |N|$. Also, let $\rho_j = w_j/p_j$ for $j \in N$.

Once a schedule σ is determined, let $C_j(\sigma)$ denote the completion time of job j for $j \in N$. When the schedule is clear from context, we omit the argument. The goal is to construct a schedule for which the total weighted completion time $\sum w_j C_j$ is minimized. This problem is denoted by P .

Let z^* denote the total weighted completion time of an optimal offline schedule for a given instance of P , i.e., an optimal schedule for an instance where all data about the jobs are known at the start of the planning horizon. In the case of equal release dates, an optimal schedule is obtained by sequencing the jobs in shortest weighted processing time (SWPT) order or, equivalently, in nonincreasing order of ρ_j (Smith [12]).

Let z^A denote the total weighted completion time delivered by some online algorithm A . We say that algorithm A is R -competitive if $z^A/z^* \leq R$ for any problem instance. Further, the minimal value of R for algorithm A is the *competitive ratio* of algorithm A .

We first present a result that restricts the class of schedules that we consider.

LEMMA 2.1. *There exists an optimal schedule in which every job starts either at the completion time of another job or at one of the possible job arrival times $t_0, \dots, t_T$.*

PROOF. Suppose that some job starts processing at time t , where $t_u < t < t_{u+1}$ for some $0 \leq u \leq T$, and the machine is idle immediately before time t . If this job starts earlier in the idle time period immediately preceding time t but not before time t_u , then the total weighted completion time does not increase. Further, any schedule of the jobs after time t in the original schedule remains feasible when the selected job starts earlier. $\square$

Henceforth, we restrict our attention to schedules characterized by Lemma 2.1.

Our analysis uses a lower bound on the value of z^* obtained through *simple job splitting relaxation*, as developed by Posner [8] and Belouadah et al. [2], and, generalized by Goemans [4] and Uma et al. [13]. This relaxation is also known as the *mean busy date relaxation*. In this relaxation, a job may be split into pieces, where the weight assigned to a piece of job j is equal to ρ_j times its processing time. Moreover, a cost is incurred on completion of each piece. The procedure described below, “Split,” optimally solves the relaxed problem. This procedure ensures that a job j with largest ρ_j among the available jobs is always being processed. A split is created at an arrival time when a new job i becomes available during the processing of job j where $\rho_i > \rho_j$.

In Procedure Split, τ is the completion time of the last scheduled job or job piece and S is the set of currently available jobs. Also, u is the index of the next potential job arrival time. Finally, z^L is the computed lower bound.

Procedure Split

Set $t_{T+1} = \infty$, $\tau = 0$, $u = 1$, $z^L = 0$, and $S = N_0$.

Set $\rho_j = w_j/p_j$ for $j \in N$.

Repeat until $u = T + 1$ and $S = \emptyset$:

If $S \neq \emptyset$:

Let job $j = \arg \max_{i \in S} \{\rho_i\}$.

If $\tau + p_j > t_u$:

Process a piece of job j in the interval $[\tau, t_u]$.

Set $p_j = p_j - (t_u - \tau)$.

Set $z^L = z^L + \rho_j(t_u - \tau)t_u + \rho_j(t_u - \tau)p_j$.

Set $\tau = t_u$, $S = S \cup N_u$, and $u = u + 1$.

Else:

Process job j from time τ .

Set $\tau = \tau + p_j$, $z^L = z^L + \rho_j p_j \tau$, and $S = S \setminus \{j\}$.

If $\tau = t_u$:

Set $S = S \cup N_u$ and $u = u + 1$.

Else:

Insert idle time in the interval $[\tau, t_u]$.

Set $\tau = t_u$, $S = N_u$, and $u = u + 1$.

We now briefly discuss Procedure Split. If $\tau + p_j > t_u$, then $p_j - (t_u - \tau)$ is the processing time of job j that is performed after time t_u . After job j is split, the first part of the job that is processed up to time t_u incurs a cost of $\rho_j(t_u - \tau)t_u$. Moreover, adding the term $\rho_j(t_u - \tau)p_j$ restores the original schedule cost (Posner [8]). Let σ^L denote the schedule delivered by Procedure Split. Let z_j^L denote the contribution of job j to z^L , resulting from the increments to z^L from the total weighted completion time and splitting cost adjustments for job j . Then, $z^L = \sum_{j \in N} z_j^L$.

The following result provides a lower bound on the optimal solution value.

LEMMA 2.2 (BELOUADAH ET AL. [2]). For problem P , $z^L \leq z^*$.

3. Lower bound on the competitive ratio. In this section, a lower bound on the competitive ratio of any online scheduling algorithm is derived. We define a problem $P(v)$ that characterizes an instance with a single job j with processing time p and the possible arrival of a second job. Job j arrives at time $t_0 = 0$. The variables are p and R , where R represents the competitive ratio. The formulation of the problem is:

$$\begin{aligned} P(v) \quad & \text{maximize } R, \\ & \text{subject to } (t_s + p)/t_{s+1} \geq R, \quad s = 0, \dots, v-1, \end{aligned} \quad (1)$$

$$(t_v + p)/(t_0 + p) \geq R, \quad (2)$$

$$p, R \geq 0.$$

Inequality (1) models the solution where job j starts at time t_s and completes at time $t_s + p > t_{s+1}$, and then another job with small processing time and large weight is released at time t_{s+1} . If job j were not in process at time t_{s+1} , then the large weight job could complete at approximately time t_{s+1} . Because this job has dominant weight, the total weighted completion time for the schedule divided by the optimal total weighted completion time is approximately $(t_s + p)/t_{s+1}$. Inequality (2) corresponds to a situation where job j starts at time t_v and no other job is released. Thus, the total weighted completion time for the schedule divided by the optimal total weighted completion time is $(t_v + p)/(t_0 + p)$.

Let $R(v)$ denote the maximum value of R for problem $P(v)$, and let $p(v)$ denote the corresponding value of p . We refer to such a solution as $(p(v), R(v))$. Further, let $R^* = \max_{1 \leq v \leq T} \{R(v)\}$. After a preliminary result, we show that R^* is a lower bound on the competitive ratio of any online algorithm.

LEMMA 3.1. $R(v) > 1$ and $p(v) > t_{s+1} - t_s$, for $0 \leq s \leq v-1$.

PROOF. The maximum value of R is equal to the minimum of the left-hand sides of constraints (1) and (2). If p is chosen so that $p > t_{s+1} - t_s$ for $s = 0, \dots, v-1$, then each of these left-hand sides exceeds 1. However, if $p \leq t_{s+1} - t_s$ for some s where $0 \leq s \leq v-1$, then R is not maximized. $\square$

We now establish the main result of this section.

THEOREM 3.1. The competitive ratio of any online scheduling algorithm is at least R^* .

PROOF. Let v denote the index for which $R^* = R(v)$. Also, let $p^* = p(v)$. Consider an instance with one or two jobs where the data for the first job are $r_1 = t_0$, $p_1 = p^*$, and $w_1 = 1$. We show that no online scheduling algorithm can generate a schedule σ for this instance for which $\sum w_j C_j(\sigma)/z^* < R^*$ is guaranteed.

In the absence of any additional job arrival, the online algorithm must make a decision about when to schedule job 1. From Lemma 2.1, we assume without loss of generality that the online algorithm starts job 1 at time t_s , where $0 \leq s \leq T$.

First, suppose that $s < v$. An adversary releases a second job at time t_{s+1} . The data for the second job are $r_2 = t_{s+1}$, $p_2 = \epsilon$, and $w_2 = W$, where $\epsilon > 0$ and $W > 0$. Because Lemma 3.1 establishes that $p_1 > t_{s+1} - t_s$, job 1 is not completed when job 2 arrives. Therefore, the online algorithm schedules job 2 so that $C_2 \geq t_s + p^* + \epsilon$. For W sufficiently large, an optimal offline algorithm schedules job 2 in the interval $[t_{s+1}, t_{s+1} + \epsilon]$ and job 1 in the interval $[t_{s+1} + \epsilon, t_{s+1} + \epsilon + p^*]$. Thus,

$$\sum w_j C_j/z^* \geq \frac{(t_s + p^*) + W(t_s + p^* + \epsilon)}{W(t_{s+1} + \epsilon) + (t_{s+1} + \epsilon + p^*)}.$$

As W becomes arbitrarily large and ϵ approaches zero, the value of $\sum w_j C_j/z^*$ approaches $(t_s + p^*)/t_{s+1} \geq R^*$, where this inequality is obtained from constraint (1) of problem $P(v)$.

Alternatively, suppose that $s \geq v$. This implies that $C_1 \geq t_v + p^*$ in the online algorithm. In this case, the adversary does not release any jobs. Therefore,

$$\sum w_j C_j/z^* \geq (t_v + p^*)/(t_u + p^*) \geq R^*,$$

where the final inequality is obtained from constraint (2) of problem $P(v)$.

Thus, regardless of the choice made by the online algorithm, $\sum w_j C_j/z^*$ can be made arbitrarily close to a value greater than or equal to R^* . Thus, no online scheduling algorithm can have a competitive ratio that is less than R^* . $\square$

The following result establishes conditions under which it is always possible to improve upon a feasible solution to problem $P(v)$. This leads to optimality conditions for $P(v)$.

LEMMA 3.2. *If (p, R) is a feasible solution to problem $P(v)$ in which either every constraint of Equation (1) is satisfied as a strict inequality or constraint (2) is satisfied as a strict inequality, then $R < R(v)$.*

PROOF. First, suppose that each constraint (1) is satisfied as a strict inequality for solution (p, R) . Consider the new solution $(p', R') = (p - \epsilon, R + \epsilon t_v/(p(p - \epsilon)))$ for some suitably small $\epsilon > 0$. Because ϵ is chosen to be sufficiently small, constraints (1) remain satisfied. Also,

$$\frac{t_v + p - \epsilon}{p - \epsilon} = \frac{t_v + p}{p} + \frac{\epsilon t_v}{p(p - \epsilon)} \geq R + \frac{\epsilon t_v}{p(p - \epsilon)},$$

where the inequality is obtained from the feasibility of (p, R) for constraint (2). Thus, the solution (p', R') is feasible for problem $P(v)$.

Alternatively, suppose that constraint (2) is satisfied as a strict inequality for solution (p, R) . Consider a solution $(p', R') = (p + \epsilon, R + \epsilon/t_v)$ for some suitably small $\epsilon > 0$. Constraints (1) remain satisfied because:

$$\frac{t_s + p + \epsilon}{t_{s+1}} \geq R + \frac{\epsilon}{t_{s+1}} \geq R + \frac{\epsilon}{t_v},$$

where the first inequality follows from the feasibility of (p, R) for constraint (1). Further, because ϵ is chosen to be sufficiently small, constraint (2) remains satisfied. Thus, solution (p', R') is feasible for problem $P(v)$.

In either case, we have constructed a feasible solution (p', R') to problem $P(v)$ in which $R' > R$. This establishes that (p, R) is not an optimal solution. Hence, $R(v) > R$. $\square$

COROLLARY 3.1. *In an optimal solution to problem $P(v)$, constraint (2) and at least one of constraints (1) are satisfied at equality.*

PROOF. Lemma 3.2 shows that if the conditions of the corollary are not satisfied, then a better solution exists. $\square$

Corollary 3.1 establishes that some constraint s of Equation (1) and constraint (2) are satisfied at equality in an optimal solution to $P(v)$. Solving $(t_s + p)/t_{s+1} = R_s(v)$ and $(t_v + p)/p = R_s(v)$ simultaneously yields:

$$R_s(v) = \left[t_{s+1} + t_s + \sqrt{(t_{s+1} - t_s)^2 + 4t_v t_{s+1}} \right] / 2t_{s+1}, \quad \text{for } s = 0, \dots, v-1. \quad (3)$$

We now show how to determine R^* efficiently.

THEOREM 3.2.

$$R^* = \max_{v=1, \dots, T} \left\{ \min_{s=0, \dots, v-1} \left\{ \left[t_{s+1} + t_s + \sqrt{(t_{s+1} - t_s)^2 + 4t_v t_{s+1}} \right] / 2t_{s+1} \right\} \right\}. \quad (4)$$

PROOF. We show that $R(v) = \min_{s=0, \dots, v-1} \{R_s(v)\}$.

Consider two feasible solutions $(p_{s_1}(v), R_{s_1}(v))$ and $(p_{s_2}(v), R_{s_2}(v))$ to problem $P(v)$ that satisfy the conditions of Corollary 3.1. For these two solutions, $(t_s + p_s(v))/t_{s+1} = R_s(v)$ and $(t_v + p_s(v))/p_s(v) = R_s(v)$, for $s = s_1, s_2$. Suppose that $R_{s_1}(v) < R_{s_2}(v)$. This implies that $p_{s_1}(v) > p_{s_2}(v)$. To complete the proof, we show that the solution $(p_{s_2}(v), R_{s_2}(v))$ is infeasible for problem $P(v)$.

The feasibility of $(p_{s_1}(v), R_{s_1}(v))$ implies:

$$(t_{s_1} + p_{s_2}(v))/t_{s_1+1} = (R_{s_1}(v)t_{s_1+1} - p_{s_1}(v) + p_{s_2}(v))/t_{s_1+1} < R_{s_2}(v).$$

Consequently, the solution $(p_{s_2}(v), R_{s_2}(v))$ does not satisfy constraint (1) for $s = s_1$, which is a contradiction. $\square$

Equation (3) shows that $R_s(v') < R_s(v)$ for $v' < v$. Therefore, Equation (4) can be calculated efficiently by first determining $R(T)$. If $s' = \arg \min_{s=0, \dots, T-1} \{R_s(T)\}$, then it is not necessary to find $R(v)$ for $v = T-1, T-2, \dots, s'$. After $R(s'-1)$ is determined, these steps can be repeated. The procedure terminates either when $R(1)$ is computed or when it is found that this value is not needed.

We provide the following bound on $R_s(v)$, which is used in Theorem 3.3.

LEMMA 3.3. $R_s(v) \geq (1 + \sqrt{5})/2$, for $s = 0, \dots, v-1$.

PROOF. Taking the derivative of Equation (3) with respect to t_s shows that $R_s(v)$ is a monotonically increasing function of t_s for $s = 0, \dots, v-1$. Moreover, if $t_s = 0$, then from Equation (3) we have:

$$R_s(v) = \left[t_{s+1} + \sqrt{t_{s+1}^2 + 4t_v t_{s+1}} \right] / 2t_{s+1} \geq (1 + \sqrt{5})/2,$$

as required. $\square$

The value of R^* plays an important role in the development of our online algorithm. We show in §5 that it also defines the competitive ratio of the algorithm. The next result provides bounds on the value of R^* .

THEOREM 3.3. $(1 + \sqrt{5})/2 \leq R^* < 2$.

PROOF. The lower bound follows from Equations (3) and (4) and Lemma 3.3.

For the upper bound, suppose that $R^* \geq 2$. Consider problem $P(v)$ for $1 \leq v \leq T$. Then, for $s = v-1$, Equation (1) implies that $t_{v-1} + p \geq 2t_v$ and Equation (2) implies that $t_v + p \geq 2t_0 + 2p$. Adding these two inequalities yields $t_{v-1} \geq 2t_0 + t_v$, which is impossible because $t_{v-1} < t_v$. Thus, $R^* < 2$, as required. $\square$

4. An online scheduling algorithm. In this section, we describe an online scheduling algorithm: Critical Shortest Weighted Processing Time (CSWPT). The algorithm selects an unscheduled job j with largest ρ_j . This job is scheduled if its processing does not extend more than a critical amount of time beyond the next potential job arrival time. Otherwise, the machine is left idle until the next potential job arrival time.

In CSWPT, τ , S , and u have the same interpretation as in Procedure Split in §2. Also, σ^C is the schedule generated by Algorithm CSWPT and z^C is the associated total weighted completion time.

Algorithm CSWPT

Set $t_{T+1} = \infty$, $\tau = 0$, $u = 1$, $z^C = 0$, and $S = N_0$.

Compute R^* using Equation (4).

Repeat until $u = T+1$ and $S = \emptyset$:

If $S \neq \emptyset$:

Let job $j = \arg \max_{i \in S} \{\rho_i\}$.

If $\tau + p_j > R^* t_u$:

Insert idle time in the interval $[\tau, t_u]$.

Set $\tau = t_u$, $S = S \cup N_u$, and $u = u+1$.

Else:

Process job j in the interval $[\tau, \tau + p_j]$.

Set $\tau = \tau + p_j$, $z^C = z^C + w_j \tau$, and $S = S \setminus \{j\}$.

If $\tau \geq t_u$:

Let $u' = \max\{v \mid t_v \leq \tau\}$.

Set $S = S \cup N_u \cup \dots \cup N_{u'}$ and $u = u' + 1$.

Else:

Insert idle time in the interval $[\tau, t_u]$.
Set $\tau = t_u$, $S = S \cup N_u$, and $u = u + 1$.

We refer to the time R^*t_u as the *critical completion time* of the job being considered for scheduling. If $\tau + p_j \leq R^*t_u$, where $j = \arg \max_{i \in S} \{\rho_i\}$, then job j can complete by the critical completion time. If job j cannot complete by R^*t_u , then the algorithm inserts idle time until t_u . When job j is processed up to time t_u or beyond, u' is defined such that $t_{u'}$ is the latest potential job arrival time up to τ . Then, jobs in $N_u \cup \dots \cup N_{u'}$ are added to the set S .

LEMMA 4.1. *Algorithm CSWPT requires $O(\max\{n \log n, T^2\})$ time.*

PROOF. Theorem 3.2 establishes that computing R^* requires $O(T^2)$ time. Because each application of the main loop either removes a job permanently from S or increments u by one, the loop is repeated $O(n + T)$ times. The unscheduled jobs in S are stored in a heap with the value of ρ as the key. Because each job is added to S and deleted from S only once, maintaining S requires $O(n \log n)$ time overall. Because the values of u' are nondecreasing, finding all such values requires $O(T)$ time overall. All the other computations within the main loop require constant time overall. Therefore, the overall time requirement of Algorithm CWSPT is $O(\max\{n \log n, T^2\})$. $\square$

5. Competitive analysis of CSWPT. In this section, we consider the competitive ratio of Algorithm CSWPT. We first consider an instance with a single job.

LEMMA 5.1. *If $n = 1$ and the job arrives at time $t_0 = 0$, then Algorithm CSWPT finds a solution that satisfies $z^C/z^L \leq R^*$.*

PROOF. Let p denote the processing time of the job. Suppose that Algorithm CSWPT schedules the job to start at some time t_v . If $v = 0$, then $z^C/z^L = 1 < R^*$.

Alternatively, if $v > 0$, then Algorithm CSWPT implies that $(t_s + p)/t_{s+1} > R^*$, for $s = 0, \dots, v - 1$. If $z^C/z^L = (t_v + p)/p > R^*$, then from Lemma 3.2, $R(v) > R^*$. This contradicts the definition of R^* . $\square$

We now show that the competitive ratio of Algorithm CSWPT is equal to R^* . From Theorem 3.1, this result is best possible. From Lemma 2.2, $z^C/z^L \geq z^C/z^*$. Therefore, in order to find an upper bound on the competitive ratio of CSWPT, we eliminate instances where the value of z^C/z^L is dominated. The following result shows that a worst-case instance has the property that $w_j = 0$ or $w_j = p_j$ for each job $j \in N$.

LEMMA 5.2. *Given any instance of problem P , there exists another instance for which z^C/z^L is at least as large and $\rho_j \in \{0, 1\}$ for $j \in N$.*

PROOF. Consider any instance of problem P with more than one distinct nonzero ρ value. Let $\rho_{\max} = \max_{j \in N} \{\rho_j\}$. For any job $k \in N$, let $E_k = \{j \in N \mid \rho_j = \rho_k\}$. Observe that available jobs are selected for scheduling in nonincreasing ρ_j order in both σ^C and σ^L . Consider any job k for which $0 < \rho_k < \rho_{\max}$.

First, consider the case where $\sum_{j \in E_k} w_j C_j(\sigma^C) / \sum_{j \in E_k} z_j^L \leq z^C/z^L$. Construct a new instance where $w_k = w_k - \epsilon$ and $w_j = w_j - \epsilon p_j/p_k$ for each $j \in E_k$ and $j \neq k$ and some $\epsilon > 0$. Note that, for this new instance, we again have $\rho_j = \rho_k$ for $j \in E_k$. By the construction of E_k , we can determine the largest ϵ such that $\epsilon \leq w_k$ and both σ^C and σ^L are unchanged by the weight adjustments (assuming that suitable tie breaking rules are applied when there are equal ρ values). Observe that the new value of z^C/z^L is at least as large as the original value. Moreover, the number of distinct nonzero ρ values is reduced by one.

Now, consider the alternative case where $\sum_{j \in E_k} w_j C_j(\sigma^C) / \sum_{j \in E_k} z_j^L > z^C/z^L$. Then, increasing w_k by ϵ and increasing the other weights w_j by $\epsilon p_j/p_k$ for $j \in E_k$ and $j \neq k$ increases z^C/z^L . Choosing the largest ϵ such that both σ^C and σ^L are unchanged, subject to the condition that the new value of ρ_k does not exceed $\rho_{\max}$, reduces the number of distinct nonzero ρ values by one.

Repetition of these arguments shows that there exists another instance for which z^C/z^L is at least as large as for the original instance and there is only one nonzero ρ value. If $\rho \neq 1$, then divide all weights by ρ . Thus, $w_j = 0$ or $w_j = p_j$ for all $j \in N$, and σ^C , σ^L , and z^C/z^L are unchanged. $\square$

As a consequence of Lemma 5.2, in the remainder of this section we restrict our attention to instances where $\rho_j = 0$ or $\rho_j = 1$ for $j \in N$. We refer to j as a *type 0 job* if $\rho_j = 0$ and as a *type 1 job* if $\rho_j = 1$. Without loss of generality, we assume that the jobs are indexed such that $r_i < r_j$ implies that $i < j$. Further, for any type 1 jobs i and j with $i < j$, job i is processed before job j by both Algorithm CSWPT and Procedure Split. Using these assumptions, we have the following properties of schedules generated by Algorithm CSWPT and Procedure Split.

Properties of schedules σ^C and σ^L .

- (a) All pieces of each type 1 job are processed contiguously in schedule σ^L .
- (b) Type 1 jobs are processed in the same sequence in schedules σ^C and σ^L .
- (c) $C_j(\sigma^L) \leq C_j(\sigma^C)$ for every type 1 job j .

Property (a) follows from the choice of job j in Procedure Split and the fact that the type 1 jobs have the largest ρ_i values. Property (b) follows from the identical choice of job j in Procedure Split and Algorithm CSWPT. Property (c) is justified as follows. First, Property (b) ensures that the type 1 jobs are sequenced in the same order in both schedules. Second, the amount of idle time inserted by Procedure Split is no greater than that inserted by Algorithm CSWPT. Finally, the amount of processing of type 0 jobs that precedes any type 1 job is no greater in σ^L than in σ^C .

To establish that Algorithm CSWPT has a competitive ratio of R^* , we prove that $C_j(\sigma^C)/C_j(\sigma^L) \leq R^*$ for each type 1 job j . From Property (a), this is sufficient to prove $z^C/z^L \leq R^*$. The following three results consider various possible configurations for job j in schedule σ^C , i.e., whether it is preceded by a type 0 job, by a type 1 job, or by idle time, respectively.

We first consider the situation where job j is processed immediately after a type 0 job i in σ^C .

LEMMA 5.3. *If a type 1 job j starts processing in σ^C when a type 0 job i completes, then $C_j(\sigma^C)/C_j(\sigma^L) < R^*$.*

PROOF. From the choice of job j in Algorithm CSWPT, job j is not available when job i starts processing. Consequently, job j is released at some time t_u during the processing of job i . Let job i start processing at time t_{l-1} or later but before time t_l , where $l \leq u$. Because jobs i and j are processed contiguously,

$$\begin{aligned} C_j(\sigma^C) &= C_i(\sigma^C) + p_j \\ &\leq R^*t_l + p_j \\ &< R^*(t_u + p_j). \end{aligned}$$

The first inequality is obtained from the condition in Algorithm CSWPT that allows job i to be processed. The second inequality uses $R^* > 1$ from Theorem 3.3 and $t_l \leq t_u$. The observation that $C_j(\sigma^L) \geq t_u + p_j$ establishes the lemma. $\square$

The next result is an “inheritance” property for the case where a type 1 job j is processed immediately after another type 1 job i in σ^C .

LEMMA 5.4. *If a type 1 job j starts processing in σ^C when another type 1 job i completes, then $C_j(\sigma^C)/C_j(\sigma^L) \leq C_i(\sigma^C)/C_i(\sigma^L)$.*

PROOF. From the properties of schedules σ^L and σ^C ,

$$C_j(\sigma^C)/C_j(\sigma^L) \leq (C_i(\sigma^C) + p_j)/(C_i(\sigma^L) + p_j) \leq C_i(\sigma^C)/C_i(\sigma^L). \quad (5)$$

The first inequality follows from Property (b) and the second inequality follows from Property (c). $\square$

Finally, we consider the situation where a type 1 job j is preceded by idle time in σ^C .

LEMMA 5.5. *If a type 1 job j starts processing in σ^C immediately after a period of idle time, then $C_j(\sigma^C)/C_j(\sigma^L) \leq R^*$.*

PROOF. Let t_u denote the start time of job j in σ^C . If $t_u = r_j$, then $C_j(\sigma^C) = C_j(\sigma^L)$ and the proof is complete. Otherwise, there must exist a period of idle time between time r_j and the start of processing of job j .

Now, consider an instance with a single type 1 job, which we denote by 0 where $r_0 = 0$ and $p_0 = C_j(\sigma^L)$. Let C'_0 denote the completion time of job 0 in the schedule delivered by Algorithm CSWPT. From Lemma 5.1,

$$C'_0 \leq R^*p_0 = R^*C_j(\sigma^L).$$

Hence, to complete the proof, it is sufficient to show that $C_j(\sigma^C) \leq C'_0$.

We first establish that $C'_0 - p_0 \geq r_j$. Let ν denote the later of t_{u-1} and the start time of the idle period that precedes t_u in σ^C . Note that $r_j \leq \nu < t_u$. Now,

$$\begin{aligned} C'_0 &\geq C_j(\sigma^L) \\ &\geq r_j + p_j \\ &\geq r_j(\nu + p_j)/\nu \\ &> r_j R^* t_u / \nu \\ &> R^* r_j. \end{aligned}$$

The fourth inequality follows from the fact that Algorithm CSWPT inserts idle time starting at time ν rather than processing job j . If $C'_0 - p_0 < r_j$, then the inequality $C'_0 > R^*r_j$ contradicts the rule that Algorithm CSWPT uses to select the start time of job 0. This establishes that $C'_0 - p_0 \geq r_j$.

For the remainder of the proof, we assume that $C'_0 < C_j(\sigma^C)$, and derive a contradiction. We first show that no type 0 jobs are processed in the interval $[C'_0 - p_0, C'_0]$ in σ^C . Because job j is type 1 and available at time r_j , the job selection procedure used by Algorithm CSWPT establishes that no type 0 job starts processing within the interval $[r_j, C_j(\sigma^C)]$ in σ^C . Thus, if a type 0 job is processed in the interval $[C'_0 - p_0, C'_0]$ in σ^C , it must be in process at time r_j . Suppose that some job h is in process at time r_j in σ^C . We show that $C_h(\sigma^C) < C'_0 - p_0$.

Let t_s denote the earliest potential job arrival time greater than the start time of job h . Thus, $t_s \leq r_j$. Let $S_j \subseteq \{1, \dots, j\}$ denote the subset of type 1 jobs that are released no earlier than time t_s , and let P_j denote the total processing time of these jobs. From the definitions of t_s and P_j ,

$$p_0 = C_j(\sigma^L) \geq t_s + P_j. \quad (6)$$

Further,

$$\begin{aligned} C_h(\sigma^C) &\leq R^*t_s \\ &< 2t_s \\ &\leq r_j + t_s \\ &\leq C'_0 - p_0 + t_s \\ &\leq C'_0 - P_j. \end{aligned} \quad (7)$$

The first inequality follows from the rule that Algorithm CSWPT uses to select the start time of job h . The second inequality follows from Theorem 3.3 and the final inequality follows from Equation (6).

Consider the interval $[C_h(\sigma^C), C'_0]$ in σ^C . Because job j is available and is of type 1, no type 0 jobs are scheduled in this interval. Further, all type 1 jobs of $\{1, \dots, j\} \setminus S_j$ are completed by the start time of job h . Thus, Algorithm CSWPT schedules only jobs of S_j and idle time within the interval $[C_h(\sigma^C), C'_0]$ in σ^C . From Equation (7),

$$C_h(\sigma^C) + P_j < C'_0.$$

Therefore, Algorithm CSWPT inserts idle time in σ^C within the interval $[C_h(\sigma^C), C'_0]$. Because $C'_0 < C_j(\sigma^C)$, this idle time precedes job j and there may be periods of idle time delaying other jobs of S_j . Let γ denote the start of the first idle time period. Also, let i denote the available job that is delayed by Algorithm CSWPT at γ in σ^C , where $i \in S_j$. Then, from the rule that Algorithm CSWPT uses to select start times, $\gamma + p_i > R^*t_v$ for some $t_v > \gamma$. Because only jobs from $S_j \setminus \{i, \dots, j\}$ are processed within the interval $[C_h(\sigma^C), \gamma]$, we have $C_h(\sigma^C) + P_j \geq \gamma + p_i$. Therefore,

$$\begin{aligned} C'_0 &> C_h(\sigma^C) + P_j \\ &\geq \gamma + p_i \\ &> R^*t_v. \end{aligned}$$

If $C'_0 - p_0 < t_v$, then the inequality $C'_0 > R^*t_v$ contradicts the rule that Algorithm CSWPT uses to select the start time of job 0. This establishes that:

$$C'_0 - p_0 \geq t_v > C_h(\sigma^C)$$

if job h is in process at time r_j in σ^C . As a result, no type 0 jobs are processed in the interval $[C'_0 - p_0, C'_0]$ in σ^C .

Now, let P'_j denote the total processing time of all type 1 jobs in the set $\{1, \dots, j\}$. We have that $p_0 \geq P'_j$ and the only jobs processed within the interval $[C'_0 - p_0, C'_0]$ in σ^C are type 1 jobs from the set $\{1, \dots, j\}$. If $C'_0 < C_j(\sigma^C)$, then there must be idle time in this interval. Let γ' denote the start of the first idle time period in $[C'_0 - p_0, C'_0]$, and let i' denote the available job that is delayed by Algorithm CSWPT at time γ' , where $i' \in \{1, \dots, j\}$. Then, from the rule that Algorithm CSWPT uses to select start times, $\gamma' + p_{i'} > R^*t_{v'}$ for some $t_{v'} > \gamma'$. Therefore,

$$\begin{aligned} C'_0 &\geq C'_0 - p_0 + P'_j \\ &\geq \gamma' + p_{i'} \\ &> R^*t_{v'}. \end{aligned}$$

The second inequality holds because within the interval $[C'_0 - p_0, \gamma')$, no jobs other than those of $\{1, \dots, i' - 1\}$ are processed. Consequently, the inequality $C'_0 > R^* t_{v'}$ implies that job 0 starts no earlier than time $t_{v'} > C'_0 - p_0$, which is a contradiction. $\square$

The main result of this section now follows.

THEOREM 5.1. *For problem P , Algorithm CSWPT has the best possible competitive ratio of R^* .*

PROOF. From Lemma 5.2, we can restrict our analysis to instances where $p_i \in \{0, 1\}$, for $i \in N$. If job i is of type 0, then it does not contribute to z^C or z^L . Thus, let job j denote the smallest index type 1 job with largest $C_j(\sigma^C)/C_j(\sigma^L)$ value.

There are only three possible configurations for scheduling job j in σ^C :

- (i) j is immediately preceded by a type 0 job,
- (ii) j is immediately preceded by a type 1 job i , and
- (iii) j is immediately preceded by a period of idle time.

Now, Lemma 5.3 establishes that $C_j(\sigma^C)/C_j(\sigma^L) \leq R^*$ for Case (i). Because $i < j$, Lemma 5.4 shows that Case (ii) does not occur. Further, Lemma 5.5 establishes that $C_j(\sigma^C)/C_j(\sigma^L) \leq R^*$ for Case (iii). Thus, $z^C/z^L = \sum_{j \in N} C_j(\sigma^C) / \sum_{j \in N} C_j(\sigma^L) \leq R^*$. From Theorem 3.1, this result is best possible. $\square$

6. Special cases. In this section, we consider two special cases of problem P . Both special cases are motivated by the observation that, in many practical situations, potential job arrival times do not occur arbitrarily. In §6.1, we assume that the intervals between potential job arrival times do not decrease rapidly. This assumption conforms to the widespread practice of short-term planning being more detailed than long-term planning. In §6.2, we consider an additional assumption about the ratio of the last two potential job arrival times, which prevents a disproportionately large interval between these two times.

6.1. Nonrapidly decreasing intervals between potential arrivals. Assume that the potential arrival times of jobs satisfy:

$$(t_{v-1} - t_{s-1}) / (t_v - t_s) \leq R(v), \quad \text{for } s = 1, \dots, v-1; v = 2, \dots, T. \quad (8)$$

The inequalities specified in Equation (8) prevent the intervals between potential job arrival times from decreasing rapidly. For this special case, the value of R^* can be computed more efficiently than for the general case considered in §3.

THEOREM 6.1. *If Equation (8) holds, then:*

$$R^* = \max_{v=1, \dots, T} \{R(v)\} = \max_{v=1, \dots, T} \left\{ \left[t_{v-1} + t_v + \sqrt{5t_v^2 + t_{v-1}^2 - 2t_{v-1}t_v} \right] / 2t_v \right\}. \quad (9)$$

PROOF. From Equation (3) and the proof of Theorem 3.2, it is sufficient to show that $R(v) = R_{v-1}(v)$ for $v = 1, \dots, T$, when an instance satisfies Equation (8).

Corollary 3.1 shows that $R(v)$ is found by the simultaneous solution of constraint s of Equation (1) for some s such that $0 \leq s \leq v-1$ and constraint (2). Thus,

$$t_s + p(v) = R(v)t_{s+1}. \quad (10)$$

Suppose that $s < v-1$. Then:

$$\begin{aligned} t_{v-1} + p(v) &= (t_s + p(v)) + (t_{v-1} - t_s) \\ &\leq R(v)t_{s+1} + R(v)(t_v - t_{s+1}) \\ &= R(v)t_v. \end{aligned} \quad (11)$$

The inequality is obtained from Equations (10) and (8).

Because constraint (1) is satisfied for $s = v-1$, we obtain $t_{v-1} + p(v) \geq R(v)t_v$. This inequality combined with Equation (11) implies that $t_{v-1} + p(v) = R(v)t_v$. As a result, constraint (1) for $s = v-1$ is satisfied at equality. Therefore, $R(v) = R_{v-1}(v)$. $\square$

Although Equation (8) is sufficient to establish Equation (9), the time required to check conditions (8) is $O(T^2)$. Thus, we restrict these conditions to:

$$(t_u - t_{u-1}) / (t_{u+1} - t_u) \leq (1 + \sqrt{5})/2, \quad \text{for } u = 1, \dots, T-1, \quad (12)$$

which can be checked more efficiently than Equation (8). To establish that Equation (12) implies Equation (8), we have:

$$\begin{aligned} t_{v-1} - t_{s-1} &= \sum_{u=s}^{v-1} (t_u - t_{u-1}) \\ &\leq ((1 + \sqrt{5})/2) \sum_{u=s}^{v-1} (t_{u+1} - t_u) \\ &\leq R(v)(t_v - t_s). \end{aligned} \quad (13)$$

The first inequality follows by applying Equation (12) for $u = s, \dots, v-1$, and the second inequality is obtained from the proof of Lemma 3.3. It follows from Equation (13) that if Equation (12) holds, then Equation (8) also holds.

COROLLARY 6.1. *If condition (12) is satisfied, then Algorithm CSWPT can be modified to run in $O(n \log n + T)$ time.*

PROOF. Observe that inequalities (12) can be checked in $O(T)$ time. From Theorem 6.1, the value of R^* can also be found in $O(T)$ time. The time requirement of the remaining steps of Algorithm CSWPT is established in Lemma 4.1. $\square$

6.2. Bounded availability time ratios. We consider a special case where, in addition to Equation (8), the largest ratio of any two consecutive potential job arrival times occurs at the end of the planning horizon. Formally, this assumption is:

$$t_{T-1}/t_T \geq t_u/t_{u+1}, \quad \text{for } u = 1, \dots, T-2. \quad (14)$$

THEOREM 6.2. *For an instance of problem P that satisfies both Equations (8) and (14),*

$$R^* = \left[t_{T-1} + t_T + \sqrt{5t_T^2 + t_{T-1}^2 - 2t_T t_{T-1}} \right] / 2t_T.$$

PROOF. Theorem 6.1 establishes that:

$$R(v) = \left[t_{v-1} + t_v + \sqrt{5t_v^2 + t_{v-1}^2 - 2t_{v-1} t_v} \right] / 2t_v, \quad (15)$$

for $v = 1, \dots, T$. Because Equation (15) is monotonically increasing in t_{v-1}/t_v over $0 < t_{v-1}/t_v < 1$, it follows from Equation (14) that $R^* = \max_{v=1, \dots, T} \{R(v)\} = R(T)$. $\square$

An important practical case that satisfies both Equations (8) and (14) occurs where the interval between successive potential job arrival times is constant over time. By rescaling the job processing times, this assumption can be written as:

$$t_{u+1} - t_u = 1, \quad \text{for } u = 0, \dots, T-1. \quad (16)$$

This assumption is motivated by the common use in business practice of an ordering process in which jobs become available periodically, for example, once a day or once a week. Observe that Equation (16) implies both Equations (8) and (14).

COROLLARY 6.2. *For an instance of problem P that satisfies Equation (16),*

$$\lim_{T \rightarrow \infty} \{R^*\} = \lim_{T \rightarrow \infty} \left\{ \left[2T - 1 + \sqrt{1 + 4T^2} \right] / 2T \right\} = 2.$$

PROOF. The result follows from Theorem 6.2. $\square$

Theorem 5.1 and Corollary 6.2 show that, as the number of planning periods increases, the competitive ratio of Algorithm CSWPT becomes arbitrarily close to 2, which is the best possible bound established by Hoogeveen and Vestjens [5] for the online problem. Thus, the advantage of knowing the potential job arrival times decreases as the number of planning periods increases.

7. Concluding remarks. This paper considers a new and practical scheduling environment that interpolates between the classical offline and online environments. Jobs arrive only at known discrete future times during the planning horizon. Moreover, the data of a job become known only at the time of its arrival. This is an *online planning period scheduling problem*, which falls within a wider class of online planning period problems.

In this environment, we consider the total weighted completion time problem. We show that a lower bound on the competitive ratio of any algorithm is the solution of a mathematical program. A closed form expression is given for the solution of this mathematical program, which provides a competitive ratio that is between $(1 + \sqrt{5})/2$ and 2, depending on the potential job arrival times. We also provide a “best possible” online algorithm with a competitive ratio that matches this lower bound. Two practically motivated special cases are considered, where the intervals between release dates do not decrease rapidly over time. For these cases, the competitive ratio is evaluated more efficiently than for the general case. When we add the further condition that the largest ratio of any two consecutive potential arrival times occurs at the end of the planning horizon, the competitive ratio asymptotically approaches the classical online ratio of 2 (Anderson and Potts [1]) as the number of potential arrival times increases.

This work suggests several interesting directions for further research. A special case that could be considered is regularly increasing intervals between the release dates. This case models the situation where there is more detailed planning in the near future than in the distant future. Another interesting issue is how much the competitive ratio can be improved by imposing restrictions on the data for arriving jobs. More generally, we can consider the design and competitive analysis of algorithms for online planning period scheduling problems with other objectives, such as minimizing the maximum lateness or maximizing the number of on-time jobs. Also, several objectives for the parallel machine environment should be considered. Another possible research direction is data-dependent analysis that estimates the benefit that a decision maker obtains from knowing the actual data of the currently available jobs. Finally, it would be interesting to study the value of information in this environment. An example would be to determine how much a decision maker should be willing to pay to obtain information about future arrivals.

Acknowledgments. The authors are grateful to two anonymous referees for their constructive suggestions, including a simplification of the proof of Lemma 5.5. This research is supported in part by National Science Foundation Grant DMI-0421823, and by the Summer Fellowship Program of the Fisher College of Business, The Ohio State University, to the first author; in part by INTAS (Grant Number 03-51-5501) to the third author; and in part by Engineering and Physical Sciences Research Council (U.K.) Grant EP/D060931/1, to all three authors.

References

- [1] Anderson, E. J., C. N. Potts. 2004. Online scheduling of a single machine to minimize total weighted completion time. *Math. Oper. Res.* **29** 686–697.
- [2] Belouadah, H., M. E. Posner, C. N. Potts. 1992. Scheduling with release dates on a single machine to minimize total weighted completion time. *Discrete Appl. Math.* **36** 213–231.
- [3] Chen, Z.-L., N. G. Hall. 2008. Maximum profit scheduling. *Manufacturing Service Oper. Management* **10** 84–107.
- [4] Goemans, M. X. 1997. Improved approximation algorithms for scheduling with release dates. *Proc. 8th ACM-SIAM Sympos. Discrete Algorithms*, New Orleans, Society for Industrial and Applied Mathematics, Philadelphia, 591–598.
- [5] Hoogeveen, J. A., A. P. A. Vestjens. 1996. Optimal online algorithms for single-machine scheduling. W. H. Cunningham, S. T. McCormick, M. Queyranne, eds. *Integer Programming and Combinatorial Optimization, Lecture Notes in Computer Science*, Vol. 1084. Springer, Berlin, 404–414.
- [6] Phillips, C., C. Stein, J. Wein. 1998. Minimizing average completion time in the presence of release dates. *Math. Programming* **82** 199–223.
- [7] Pinedo, M. L. 2002. *Scheduling: Theory, Algorithms and Systems*, 2nd ed. Prentice Hall, Englewood Cliffs, NJ.
- [8] Posner, M. E. 1985. Minimizing weighted completion times with deadlines. *Oper. Res.* **33** 562–574.
- [9] Pruhs, K., J. Sgall, E. Torng. 2004. Online scheduling. J. Y.-T. Leung, ed. *Handbook of Scheduling: Algorithms, Models and Performance Analysis*, Chapter 15. CRC Press, Boca Raton, FL, 1–43.
- [10] Reid, R. D., N. R. Sanders. 2005. *Operations Management: An Integrated Approach*, 2nd ed. Wiley, Hoboken, NJ.
- [11] Russell, R. S., B. W. Taylor, III. 2006. *Operations Management: Quality and Competitiveness in a Global Environment*, 5th ed. Wiley, Hoboken, NJ.
- [12] Smith, W. E. 1956. Various optimizers for single-stage production. *Naval Res. Logist. Quart.* **3** 59–66.
- [13] Uma, R. N., J. Wein, D. P. Williamson. 2006. On the relationship between combinatorial and LP-based lower bounds for NP-hard scheduling problems. *Theoret. Comput. Sci.* **361** 241–256.